

1. Introducción

Las pruebas automatizadas permiten comprobar que una aplicación funciona de acuerdo con lo esperado y ayudan a detectar regresiones cuando el código evoluciona. En el ecosistema React, Jest y React Testing Library (RTL) son herramientas ampliamente utilizadas para construir y ejecutar pruebas de componentes y de la lógica que los acompaña.

2. ¿Qué es Jest?

Jest es un framework de testing para JavaScript desarrollado originalmente en Meta. Permite crear, organizar y ejecutar pruebas automatizadas. Aunque puede utilizarse en distintos proyectos JavaScript, se integra especialmente bien con aplicaciones React.

Características principales de Jest

- **Runner de pruebas:** descubre y ejecuta los archivos de test.
- **Matchers:** permite expresar expectativas como `expect(valor).toBe(...)`, `toEqual(...)`, `toHaveLength(...)`, etc.
- **Mocks y funciones simuladas:** facilita reemplazar dependencias o funciones durante una prueba.
- **Spies:** permite observar si una función fue llamada, cuántas veces y con qué argumentos.
- **Cobertura de código:** puede generar información sobre qué partes del código fueron ejercitadas por las pruebas.
- **Configuración y ecosistema:** dispone de una amplia integración con proyectos JavaScript y React.

¿Para qué sirve? Principalmente, Jest sirve para verificar automáticamente comportamientos del software. Por ejemplo, se puede comprobar que una función devuelve el resultado correcto, que una interacción modifica el estado esperado o que un componente produce determinados resultados.

3. ¿Qué es React Testing Library?

React Testing Library es una biblioteca orientada a probar componentes React desde la perspectiva de su usuario. En lugar de centrarse en detalles internos de implementación —como estados privados o métodos internos— promueve consultar y utilizar la interfaz de manera similar a como lo haría una persona.

Características principales de React Testing Library

- **Renderizado de componentes:** permite montar un componente React dentro de un entorno de prueba.
- **Consultas orientadas al usuario:** facilita buscar elementos por rol, texto, etiqueta, placeholder u otros atributos.
- **Simulación de interacciones:** permite comprobar clics, escritura y otras acciones del usuario.
- **Enfoque en comportamiento:** las pruebas verifican qué puede observar y hacer el usuario.
- **Accesibilidad:** las consultas por roles y nombres accesibles fomentan interfaces más accesibles.
- **Integración con Jest:** RTL puede utilizarse junto con Jest para definir expectativas y ejecutar la suite.

¿Para qué sirve? RTL sirve para comprobar que un componente React se comporta correctamente desde el punto de vista de la interfaz. Por ejemplo, se puede verificar que aparece un botón, que al hacer clic se incrementa un contador y que el nuevo valor se muestra en pantalla.

4. ¿Cómo trabajan juntos Jest y React Testing Library?

Jest y React Testing Library cumplen funciones complementarias. RTL proporciona las herramientas para renderizar el componente y buscar/interactuar con sus elementos; Jest proporciona el entorno de ejecución de las pruebas, las expectativas (matchers), mocks y otras capacidades de testing.

Una forma sencilla de pensarlo es: RTL ayuda a interactuar con la interfaz; Jest ayuda a comprobar que el resultado de esa interacción coincide con lo esperado.

5. Ejemplo sencillo en React

A continuación se presenta un componente contador. El usuario puede hacer clic en un botón para incrementar el valor. Luego se crea una prueba que comprueba tanto el valor inicial como el comportamiento del botón.

5.1. Componente React

```
import { useState } from 'react';

export default function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h1>Contador</h1>
      <p>Valor: {count}</p>

      <button onClick={() => setCount(count + 1)}>
        Incrementar
      </button>
    </div>
  );
}
```

5.2. Prueba con Jest y React Testing Library

```
import { render, screen } from '@testing-library/react';
import { fireEvent } from '@testing-library/react';
import Counter from './Counter';

test('incrementa el contador al hacer clic', () => {
  render(<Counter />);

  // Estado inicial
  expect(screen.getByText('Valor: 0')).toBeInTheDocument();

  // Interacción del usuario
  const button = screen.getByRole('button', {
    name: 'Incrementar'
  });

  fireEvent.click(button);

  // Resultado esperado
  expect(screen.getByText('Valor: 1')).toBeInTheDocument();
});
```

5.3. Explicación paso a paso

1. **render(<Counter />):** React Testing Library renderiza el componente dentro del entorno de pruebas.
2. **screen.getByText('Valor: 0'):** La prueba busca en la interfaz el texto que representa el estado inicial del contador.

3. **expect(...).toBeInTheDocument():** Jest verifica que ese elemento existe en el documento renderizado.
4. **screen.getByRole('button', { name: 'Incrementar' }):** RTL localiza el botón utilizando su rol accesible y su nombre visible.
5. **fireEvent.click(button):** Se simula una interacción equivalente a hacer clic sobre el botón.
6. **expect(screen.getByText('Valor: 1')).toBeInTheDocument():** Finalmente, Jest comprueba que después del clic aparece el nuevo valor.

6. Configuración mínima del proyecto

En un proyecto React que utilice Jest y React Testing Library, normalmente se instalan los paquetes necesarios del ecosistema de testing. La configuración concreta depende de si el proyecto utiliza Vite, Create React App, Next.js u otra herramienta de construcción.

```
npm install -D jest @testing-library/react @testing-library/jest-dom
npm install -D @testing-library/user-event
```

Nota: en proyectos React modernos, la herramienta de build puede proporcionar o recomendar un runner de pruebas distinto. El ejemplo conceptual sigue siendo válido: RTL se ocupa de probar la interfaz y Jest puede encargarse de ejecutar las pruebas y realizar las aserciones.

7. Jest vs. React Testing Library

Aspecto	Jest	React Testing Library
Función principal	Ejecutar y organizar pruebas	Probar componentes React
Aserciones	Sí, mediante expect y matchers	Se apoya en Jest y/o otros matchers
Renderizado React	No es su objetivo principal	Sí
Interacción con UI	Puede apoyar mediante mocks/utilidades	Sí, orientada al comportamiento del usuario
Mocks y spies	Sí	No es su función principal
Enfoque	Framework/runner de testing	Biblioteca de testing de UI

8. Buenas prácticas

- Probar comportamientos observables por el usuario y evitar depender innecesariamente de detalles internos.
- Preferir consultas accesibles, como `getByRole`, cuando sea posible.
- Mantener cada prueba enfocada en un comportamiento concreto.
- Usar nombres de pruebas descriptivos que expliquen qué comportamiento se verifica.
- Evitar pruebas excesivamente acopladas a la estructura interna del componente.
- Complementar las pruebas unitarias/de componentes con otras pruebas apropiadas para la aplicación.

9. Conclusión

Jest y React Testing Library forman una combinación útil para automatizar pruebas en aplicaciones React. Jest aporta el entorno para ejecutar pruebas, realizar aserciones y trabajar con mocks, mientras que React Testing Library proporciona herramientas para renderizar componentes y simular una interacción cercana a la experiencia real del usuario. El ejemplo del contador demuestra el flujo fundamental: renderizar el componente, localizar un elemento de la interfaz, interactuar con él y verificar el resultado esperado.